

Deploy in Cloud — Hotel Excelsior

Corso: Applicazioni Web, Mobile e Cloud — A.A. 2025/2026

Autori: Riccardo Di Lizio, Alberto Terenzi

URL pubblico: <https://hotel-excelsior.uk>

Repository: <https://github.com/riccardodilizio-art/Progetto-Applicazioni-Web-Mobile-e-Cloud>

1. Introduzione

Il Modulo 2 del progetto Hotel Excelsior ha avuto come obiettivo portare in produzione l'applicazione sviluppata durante il Modulo 1, fino a quel momento eseguibile soltanto in locale tramite Docker Compose. L'esigenza era quella di rendere il sito raggiungibile da Internet tramite un URL pubblico, servito in HTTPS con un certificato browser-trusted, e di automatizzare il processo di rilascio in modo che ogni modifica al codice venisse propagata in produzione senza intervento manuale.

Per rispettare i vincoli di budget tipici di un progetto universitario, è stato adottato uno stack misto che combina servizi gratuiti e a basso costo, sfruttando deliberatamente i tre modelli di servizio cloud studiati a lezione (IaaS, PaaS, SaaS) all'interno di un'unica architettura. Il costo complessivo del deploy si attesta sotto i 5€ per il mese della consegna, a fronte di un'esperienza didattica completa che attraversa containerizzazione, reverse proxy, terminazione TLS, tunneling sicuro, pipeline CI/CD e considerazioni di scalabilità.

2. Architettura

L'applicazione è ospitata su una macchina virtuale Linux fornita da Hetzner Cloud, dove gira l'intero stack containerizzato attraverso Docker Compose. Il traffico HTTPS proveniente da Internet non raggiunge direttamente la VPS: viene invece intercettato da Cloudflare Edge, terminato sui loro server con un certificato

Universal SSL gestito automaticamente, e inoltrato alla macchina virtuale attraverso un tunnel outbound persistente. Sulla VPS, nginx fa da reverse proxy verso i due container applicativi: il frontend React (servito come build statico tramite un'istanza nginx interna al container) e il backend ASP.NET Core. La persistenza dei dati relazionali è affidata a Neon, un servizio managed di PostgreSQL già adottato durante il Modulo 1.

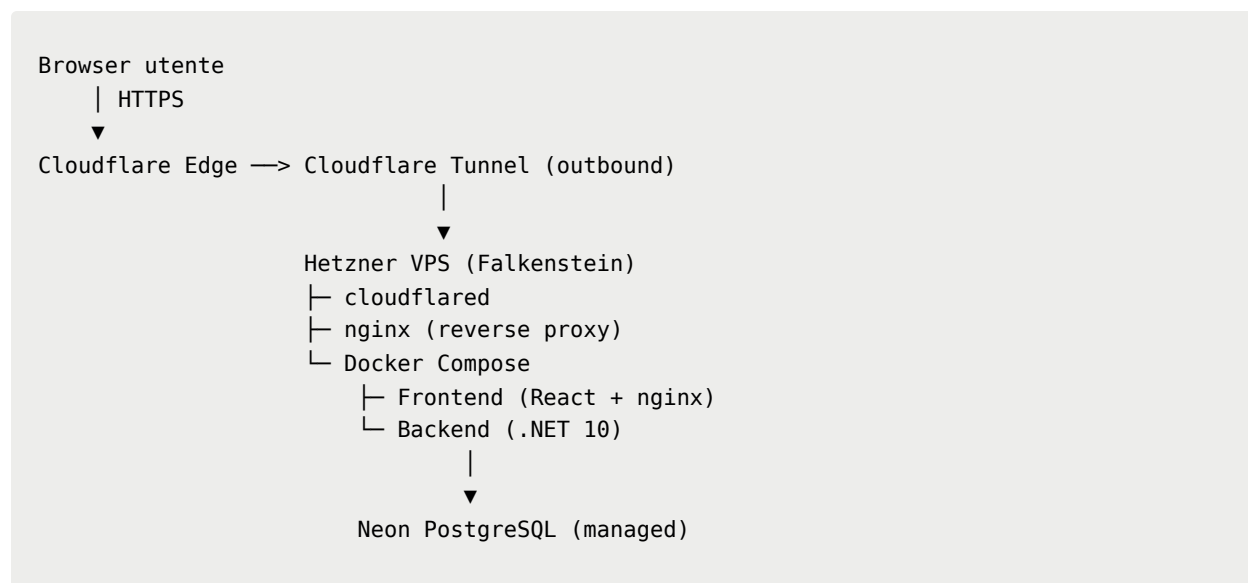


Figura 1 — Architettura del deploy

3. Tecnologie utilizzate

Hetzner Cloud (IaaS)

La componente compute è stata realizzata con una VPS Hetzner Cloud modello CX23, una macchina virtuale dotata di 2 vCPU, 4 GB di RAM e 40 GB di storage SSD, ospitata nel datacenter di Falkenstein in Germania. La scelta di Hetzner è motivata principalmente dal rapporto qualità/prezzo: a circa 4€/mese si ottengono risorse abbondanti per il workload del progetto, con datacenter europei che riducono la latenza verso utenti italiani. Trattandosi di un servizio IaaS classico, la responsabilità di configurare il sistema operativo, installare il runtime, gestire firewall e aggiornamenti ricade interamente sullo sviluppatore. La VPS è stata configurata partendo da un'immagine Ubuntu Server 24.04 LTS, su cui è stato applicato un set di pratiche di hardening: disabilitazione del login SSH come root, creazione di un utente non privilegiato dedicato al deploy, autenticazione SSH

esclusivamente tramite chiavi pubbliche e firewall UFW configurato in modalità "default deny" con apertura solo delle porte strettamente necessarie.

Docker e Docker Compose

Tutta l'applicazione viene eseguita in container, orchestrati tramite Docker Compose. La scelta della containerizzazione garantisce che l'ambiente di esecuzione in produzione sia identico a quello di sviluppo locale, eliminando di fatto la classica problematica del "funziona sul mio PC". Il file `docker-compose.yml` definisce due servizi: il frontend, che è una build statica React generata con Vite e servita da un'istanza nginx all'interno del container, e il backend, che esegue l'API ASP.NET Core compilata in modalità Release. I container comunicano attraverso una rete privata interna gestita da Docker, mentre due volumi persistenti garantiscono che le immagini caricate dall'amministratore (`uploads/`) e le chiavi di DataProtection sopravvivano ai riavvii dei container.

nginx come reverse proxy

Davanti ai container è stata installata un'istanza di nginx direttamente sulla VPS host (non in container), che svolge la funzione di reverse proxy. Tutte le richieste in arrivo sulla porta 80 vengono inoltrate al container frontend, che gestisce sia i file statici della SPA sia il proxy verso il backend per le chiamate API. Questa architettura a doppio livello (nginx host + nginx container) è stata adottata per separare nettamente la gestione della rete dall'applicazione: l'host nginx si occupa di routing, header forwarding e limiti di upload, mentre il container è completamente isolato dal mondo esterno e può essere ricostruito senza toccare la configurazione di rete. Per garantire il corretto funzionamento dietro un reverse proxy, l'applicazione ASP.NET Core include il middleware `ForwardedHeaders`, che ricostruisce le informazioni originali del client (IP e protocollo) leggendo gli header `X-Forwarded-For` e `X-Forwarded-Proto`.

Cloudflare Tunnel (PaaS)

Per esporre il sito su Internet con un dominio personalizzato e HTTPS valido, dopo aver scartato la soluzione classica Let's Encrypt per ragioni tecniche illustrate nel paragrafo di approfondimento, è stato adottato **Cloudflare Tunnel**. Questo servizio crea un canale di comunicazione persistente tra la VPS e l'edge network di Cloudflare, tramite il quale viene instradato tutto il traffico in entrata. Il dominio

`hotel-excelsior.uk` è stato registrato attraverso il servizio Cloudflare Registrar a un costo di circa 7€ annui, e il certificato HTTPS è fornito automaticamente da Cloudflare tramite il programma Universal SSL.

Neon PostgreSQL (SaaS)

La persistenza relazionale è affidata a Neon, una soluzione SaaS che fornisce PostgreSQL come servizio managed con un piano gratuito sufficiente per il progetto. Neon si occupa di backup automatici, alta disponibilità e patch di sicurezza, eliminando completamente la gestione del database dal carico operativo dello sviluppatore. La stessa istanza viene utilizzata sia in sviluppo locale sia in produzione, semplificando il workflow del team. Connessione e credenziali sono iniettate nell'applicazione tramite variabili d'ambiente lette dal file `.env`, che è escluso dal versionamento.

GitHub Actions (CI/CD)

L'automazione del rilascio è gestita da una pipeline GitHub Actions che si attiva a ogni push sul branch `main`. La pipeline è strutturata in tre job sequenziali: un primo job compila e analizza staticamente il frontend con `npm run lint` e `npm run build`; un secondo compila il backend e ne esegue la suite di test xUnit composta da 13 test (preservata dal Modulo 1); infine, solo se entrambi i job precedenti hanno successo, un terzo job si connette via SSH alla VPS Hetzner ed esegue il rebuild dei container Docker. L'autenticazione SSH avviene tramite una chiave dedicata, generata appositamente per il workflow e memorizzata in GitHub Secrets, separata dalle chiavi personali degli sviluppatori per applicare il principio del "least privilege". Il deploy completo, dal push fino al sito aggiornato, richiede mediamente 3-4 minuti.

4. Approfondimento: perché Cloudflare Tunnel

L'introduzione di Cloudflare Tunnel rappresenta la scelta più caratterizzante dell'intera architettura e merita un approfondimento, sia perché risolve un problema reale incontrato durante il deploy, sia perché illustra concretamente come un servizio PaaS possa semplificare significativamente le sfide infrastrutturali.

Inizialmente è stata tentata la strada "classica" per ottenere HTTPS su una VPS: utilizzare un sottodominio gratuito offerto da DuckDNS e richiedere un certificato a Let's Encrypt tramite il client Certbot. Questa configurazione, perfettamente documentata e ampiamente utilizzata nella community, ha funzionato sul piano del routing HTTP ma è fallita durante la fase di validazione del certificato. Let's Encrypt, dal 2024, applica un meccanismo di **multi-perspective validation** che verifica il controllo del dominio interrogando i server DNS da più datacenter sparsi geograficamente nel mondo. I server DNS di DuckDNS, sotto carico, mostravano timeout intermittenti su alcuni di questi datacenter, facendo fallire la validazione complessiva nonostante il dominio fosse formalmente raggiungibile.

Dopo alcuni tentativi infruttuosi, è stata adottata una strategia completamente diversa basata su Cloudflare Tunnel. Il principio di funzionamento è particolarmente elegante: invece di esporre porte HTTPS pubbliche sulla VPS, è la VPS stessa a iniziare una connessione **outbound** verso l'infrastruttura Cloudflare, mantenendola aperta in modo persistente. Quando un browser richiede <https://hotel-excelsior.uk>, la richiesta raggiunge prima un server edge di Cloudflare, che termina la connessione TLS utilizzando un certificato gestito automaticamente, e la inoltra alla VPS attraverso il tunnel pre-esistente.

Questo approccio comporta vantaggi multipli rispetto al modello tradizionale:

Dal punto di vista della **sicurezza**, la VPS non espone più alcuna porta pubblica per il traffico web. La superficie d'attacco si riduce drasticamente: tecnicamente, sarebbe possibile chiudere completamente le porte 80 e 443 sul firewall lasciando esposta solamente la porta 22 per la gestione SSH. Eventuali tentativi di port scanning o attacchi diretti all'IP della VPS troverebbero un firewall che rifiuta ogni connessione.

Dal punto di vista **operativo**, la gestione del certificato HTTPS è completamente automatizzata da Cloudflare: non è necessario configurare Certbot, gestire scadenze, programmare rinnovi o intervenire in caso di problemi di validazione. Lo sviluppatore può concentrarsi sull'applicazione, mentre l'infrastruttura TLS è un dettaglio invisibile.

Dal punto di vista delle **performance**, Cloudflare offre gratuitamente la sua content delivery network globale, che cache i contenuti statici (immagini, CSS, JavaScript) sui server edge più vicini all'utente finale. La protezione DDoS è inclusa di default in tutti i piani, anche quelli gratuiti.

L'unico costo concreto di questa soluzione è la dipendenza da un servizio esterno: se Cloudflare avesse un disservizio, il sito risulterebbe irraggiungibile. Si tratta tuttavia di un compromesso ragionevole considerando l'affidabilità storica dell'azienda e i numerosi vantaggi sopra elencati. In un contesto enterprise con esigenze di indipendenza tecnologica si potrebbero valutare alternative come Nginx Plus con automazione Let's Encrypt o servizi gestiti equivalenti su Azure (Front Door) o AWS (CloudFront), ma per un progetto didattico Cloudflare Tunnel rappresenta probabilmente il miglior rapporto tra semplicità implementativa, sicurezza e funzionalità.

5. Considerazioni finali

Il deploy realizzato copre tutti gli aspetti fondamentali di un'applicazione cloud-native moderna: containerizzazione, terminazione HTTPS gestita, pipeline CI/CD automatizzata, separazione dei segreti dal codice sorgente, e un database remoto. L'architettura è stata progettata con principi di sicurezza a strati (defense in depth) e di automazione, riducendo il rischio di errori manuali e i tempi di rilascio.

Pur essendo una soluzione single-node che presenta un single point of failure nella singola VPS, l'applicazione è già strutturata per essere scalata orizzontalmente: il backend è stateless grazie all'uso dei token JWT, le sessioni non sono mantenute in memoria, il database è esterno, e i secrets sono gestiti tramite variabili d'ambiente. Per evolvere verso una soluzione altamente disponibile sarebbe sufficiente aggiungere un secondo nodo dietro un load balancer Hetzner e spostare lo storage delle immagini su un servizio object storage S3-compatibile, modifiche che richiederebbero giorni e non settimane di lavoro.

Tra i possibili miglioramenti futuri, oltre alla scalabilità orizzontale già citata, si segnalano l'introduzione di un sistema di monitoring strutturato (ad esempio Prometheus + Grafana, oppure Uptime Kuma per soluzioni più leggere), la formalizzazione dell'infrastruttura tramite Infrastructure as Code (Terraform), e l'adozione di un workflow di branching più articolato con ambiente di staging per testare le modifiche prima della promozione in produzione.

Riferimenti

- Hetzner Cloud — <https://www.hetzner.com/cloud>
- Cloudflare Tunnel — <https://developers.cloudflare.com/cloudflare-one/connections/connect-networks/>
- Docker — <https://docs.docker.com/>
- Neon PostgreSQL — <https://neon.tech/docs>
- The Twelve-Factor App — <https://12factor.net/>
- NIST Definition of Cloud Computing (SP 800-145)